

第九章 解码与部署

当完成训练后，我们就可以将大语言模型部署到真实场景中进行使用。大语言模型是通过文本生成的方式进行工作的。在自回归架构中，模型针对输入内容（即提示文本，详见第 10 章）逐个单词生成输出内容的文本。这个过程一般被称为解码。在本章的内容中，我们将首先介绍常见的解码策略（第 9.1 节）以及相应的优化加速算法（第 9.2 节），然后介绍对大语言模型大小进行压缩（第 9.3 节）以适应低资源场景。

9.1 解码策略

大语言模型的生成方式本质上是一个概率采样过程，需要合适的解码策略来生成合适的输出内容。下面将介绍大语言模型的常见解码策略。

9.1.1 背景

在介绍具体的解码策略之前，首先介绍一下语言模型解码的背景知识。这里，主要介绍自回归场景下的解码策略。算法 2 展示了一个整体的自回归解码流程。可以看到，模型 M 每次根据当前上下文词元序列 $u = [u_1, u_2, \dots, u_t]$ 建模下一个词的概率分布 P （即公式 5.11 的输出），然后根据一定的解码策略选择下一个词 u' ，之后再将 u 和 u' 作为新的上下文重复上述步骤，直到生成结束词元或者达到长度上限为止。在这个流程中，解码策略将主要关注如何基于概率分布 P 选择合适的下一个词 u' 。自回归的解码策略并不局限于特定架构，常见的编码器-解码器、因果解码器和前缀解码器均可适用。

算法 2 自回归解码流程

输入：模型 M ，输入词元序列 u

输出：输出词元序列 y

- 1: **repeat**
 - 2: $P = M(u)$ # 生成下一个词元的概率分布
 - 3: $u' \sim P$ # 从分布中采样得到下一个词元
 - 4: $u \leftarrow u \oplus [u']$
 - 5: **until** u' 是结束词元或者 u 的长度超过预设长度.
 - 6: $y \leftarrow u$
-

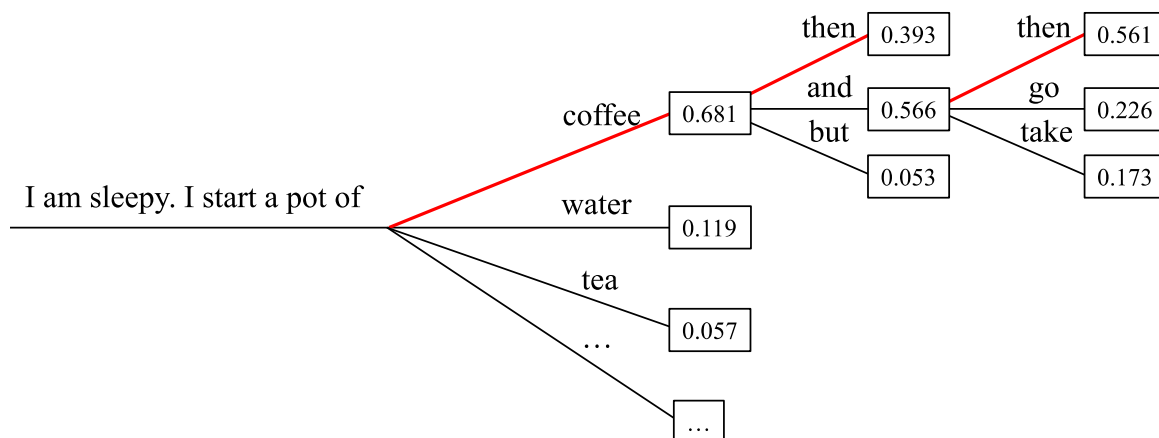


图 9.1 贪心搜索示意图

回顾第 6.1 节的内容，目前常见的大语言模型主要是通过语言建模任务（参见公式 6.1）进行预训练的。基于这种训练方式，一种直观的解码策略是贪心搜索（Greedy Search）。具体来说，贪心搜索在每个生成步骤中都选择概率最高的词元，其可以描述为以下形式：

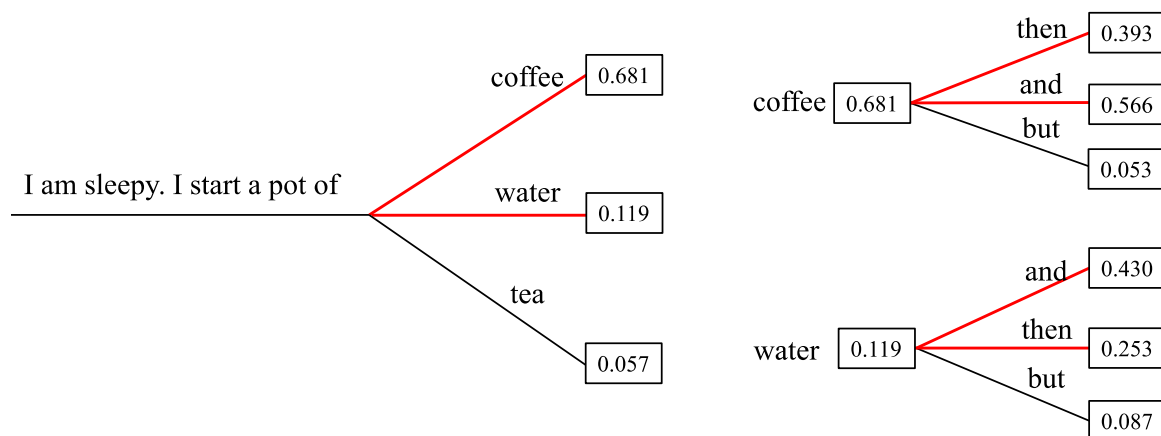
$$u_i = \arg \max_u P(u|u_{<i}). \quad (9.1)$$

图 9.1 给出了一个贪心搜索的例子。在预测句子 “*I am sleepy. I start a pot of*” 的下一个词元时，贪心搜索选择了当前步骤下概率最高的词元——“coffee”。然后模型将 “coffee” 加入上下文，并不断重复该过程。由于贪心搜索所采取的是确定性策略，它的效果在不同类型的任务中具有一定的差异。在机器翻译和文本摘要等任务中，任务输出高度依赖于输入内容，贪心搜索通常能够获得不错的结果，但是在开放式生成任务（如故事生成和对话系统）中，贪心搜索有时会因为过于关注局部最优，而生成不自然、重复的句子 [126]。

除了贪心搜索外，另一种可选的解码策略是概率采样（Probability Sampling）。该方法根据模型建模的概率分布采样得到下一个词元，旨在增强生成过程中的随

I am sleepy. I start a pot of _____					
coffee	0.681	strong	0.008	soup	0.005
water	0.119	black	0.008	...	...
tea	0.057	hot	0.007	happy	4.3e-6
rice	0.017	oat	0.006	Boh	4.3e-6
chai	0.012	beans	0.006	...	...

图 9.2 “*I am sleepy. I start a pot of*” 语境中下一个词元的降序排列概率分布（图片来源：[10]）

图 9.3 束搜索示意图 ($n=2$)

机性和结果的多样性:

$$u_i \sim P(u|u_{<i}). \quad (9.2)$$

在图 9.2 中展示了下一个词元的概率分布，虽然单词“coffee”被选中的概率较高，但基于采样的策略同时也为选择其他单词（如“water”、“tea”等）留有一定的可能性，从而增加了生成文本的多样性和随机性。

9.1.2 贪心搜索的改进

贪心搜索在每个生成步骤中均选择最高概率的词元，这可能会由于忽略在某些步骤中概率不是最高、但是整体生成概率更高的句子而造成局部最优。为了解决这个问题，可以进一步采用以下的改进策略。

- 束搜索. 在解码过程中，束搜索 (Beam Search) [236] 会保留前 n 个具有最高概率的句子，并最终选取整体概率最高的生成回复。这里的 n 被称为束大小 (Beam Size)。当 $n = 1$ ，束搜索就退化为贪心搜索。如图 9.3 所示 ($n = 2$)，第一步保留了概率最高的两个词“coffee”和“water”作为候选；第二步基于“coffee”和“water”均进行扩展，得到模型在两个上下文内容下的概率分布，最后选择联合概率最高的两个句子“coffee then”和“coffee and”作为候选。在下面的生成步骤中，将会继续基于这两个候选去进行扩展，每次都选择联合概率最高的两个句子。最后，当两个束的句子均生成结束后，选择整体生成概率最高的候选句子作为最终的输出。在实践中，束的数量通常设定在 3 到 6 的范围内，设置过大的束会显著增加运算开销，并可能会导致性能下降。

- 长度惩罚. 由于束搜索中需要比较不同长度候选句子的概率，往往需要引入

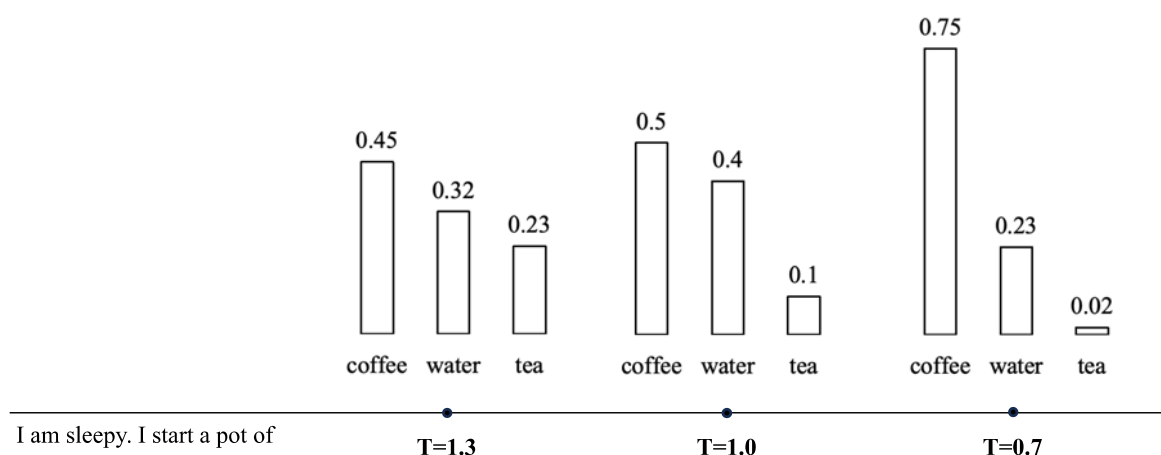


图 9.4 温度设置为 1.3、1.0 和 0.7 时的下一个词的概率分布变化

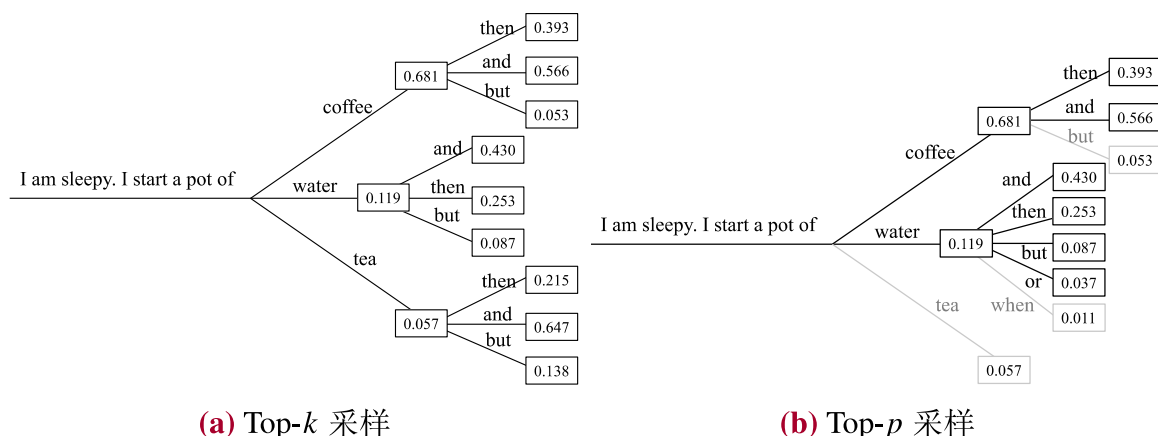
长度惩罚（Length Penalty）（亦称为长度归一化）技术。如果没有长度惩罚，传统的束搜索会倾向于生成较短的句子，因为每生成一个单词，都会乘以一个小于 1 的概率，使得句子的生成概率逐渐变小。因此，可以在生成概率的计算中引入长度惩罚，通过将句子概率除以其长度的指数幂 α ，对于句子概率进行归一化处理，从而鼓励模型生成更长的句子。在实践中， α 通常设置为 0.6 到 0.7 之间的数值。

- 重复惩罚. 为了缓解贪心搜索重复生成的问题，可以使用 n -元惩罚（ n -gram Penalty）来强制避免生成重复的连续 n 个词元，实践中 n 通常设置为 3 到 5 之间的整数。进一步地，研究人员还提出了相对“温和”的惩罚机制来降低生成重复词元的概率，而不是“一刀切”地完全避免某些短语的生成，如出现惩罚（Presence Penalty）和频率惩罚（Frequency Penalty）。具体地，出现惩罚在生成过程中会将已经生成词元的 logits（公式 5.11）减去惩罚项 α 来降低该词元之后生成的概率。频率惩罚相较于出现惩罚，会记录每个词元生成的数目，然后减去出现次数乘以惩罚项 α ，因此如果一个词元生成得越多，惩罚也就越大。在实践中， α 的取值范围通常在 0.1 到 1 之间。这些重复惩罚方法不止适用于贪心搜索，对于随机采样也均适用。

9.1.3 随机采样的改进策略

基于概率采样的方法会在整个词表中选择词元，这可能会导致生成不相干的词元（例如图 9.2 中的“happy”和“Boh”）。为了进一步提高生成质量，可以进一步使用一些改进的采样策略，减少具有极低概率词汇对于生成结果的影响。

- 温度采样（Temperature Sampling）. 为了调节采样过程中的随机性，一种有

图 9.5 Top- k 采样和 Top- p 采样示意图

效的方法是调整 softmax 函数中的温度系数。具体来说，公式 5.11 中的 $\mathbf{l} = \mathbf{W}^L \mathbf{y}_L$ 称为 logits，调整温度系数后的 softmax 计算式如下：

$$P(u_j | \mathbf{u}_{<i}) = \frac{\exp(l_j/t)}{\sum_{j'} \exp(l_{j'}/t)}, \quad (9.3)$$

其中， $l_{j'}$ 表示每个候选词元的 logit， t 是温度系数。具体来说，降低温度系数 t 会使得概率分布更加集中，从而增加了高概率词元的采样可能性，同时降低了低概率词元的采样可能；当温度系数 t 设置为 1 时，该公式退化为标准的随机采样方法；而当 t 趋近于 0 时，实际上等同于贪心搜索，即总是选择概率最高的词。此外，当 t 趋近于无穷大时，温度采样会退化为均匀采样。图 9.4 展示了在温度系数分别设置为 1.3、1.0 和 0.7 时下一个词元的概率分布情况。

- **Top- k 采样 (Top- k Sampling)**. 与温度采样不同，top- k 采样策略是直接剔除概率较低的词元，限制模型从概率最高的前 k 个词元中进行采样 [237]。以图 9.5 (a) 中所示的生成过程为例，当采用 top-3 采样策略时，模型将会从“coffee”、“water”、“tea”这三个概率最高的词元中，基于原始概率分布进行采样。

- **Top- p 采样 (Top- p Sampling)**. 由于 top- k 采样策略并不考虑整体概率分布，因此固定的常数 k 可能无法适应不同的上下文语境。在较为确定的生成场景中，例如给定前缀为“世界最高峰是”，当 k 设置为大于 1 的数值时，均有可能引入错误答案；而在需要多样性的场景中，例如“我最喜欢的城市是”， k 设置为较小的值则会限制模型的多样化生成。为了克服上述问题，研究人员提出了 top- p 采样方法（又称为核采样，Nucleus Sampling）[126]。该方法的核心思想是从一个符合特定概率条件的最小词元集合中进行采样，要求其中包含的所有词元累积概率大于或等于预设阈值 p 。在具体的实现过程中，top- p 采样首先会按照生成概率从高到低的

顺序对词元进行排序，然后不断将词元添加到一个临时的集合中，直到集合的累积概率首次超过阈值 p 。图 9.5 (b) 中展示了 top- p 采样的解码流程 ($p = 0.8$)：第一步时 “coffee” 和 “water” 的累积概率为 $0.681 + 0.119 = 0.8$ ，因此只会在这两个单词中进行采样，而不会考虑 “tea”。

- **对比解码 (Contrastive Decoding)**。相关研究表明 [238]，由于大模型比小模型具有更强的生成能力，因而在预测下一个词元时，大语言模型相较于小模型更倾向于为重要词元分配更高的概率。基于这个想法，对比解码通过计算一个较大的语言模型（例如 GPT-2 XL）和一个较小的语言模型（例如 GPT-2 small）之间的对数概率分布差值，然后基于归一化的差值分布采样下一个词元，从而有效地提升重要词元在生成过程中的影响力。为了方便读者理解，这里构造一个例子来说明对比解码的工作原理。在预测一个给定片段“李时珍是湖北人，他出生于__”的下一个词时，GPT-2 XL 有 15% 的概率生成“湖北”、10% 的概率生成“明朝”，而 GPT-2 small 有 10% 的概率生成“湖北”、0.1% 的概率生成“明朝”，可以看到虽然 GPT-2 XL 生成“湖北”的概率仍然最高，但是其生成“明朝”的概率大幅增长，对比解码可以有效利用这一现象，在解码过程中提升重要词汇的影响力。

9.1.4 实际使用设置

在实践中，现有的代码库（如 Transformers）和大语言模型的公开 API（例如 OpenAI）都集成了多种解码策略，以适应不同的文本生成任务。下面介绍几个代表性的大语言模型的解码设置：

- **T5**. T5 默认采用贪心搜索策略。在翻译和摘要任务中，它使用束搜索（束大小为 4）并结合长度惩罚（惩罚因子为 0.6）来优化生成结果。

- **GPT-3**. GPT-3 在所有生成任务中都使用束搜索（束大小为 4）和长度惩罚（惩罚因子为 0.6）。

- **Alpaca**. Alpaca 主要面向开放式生成任务，因此使用了基于采样的策略，包括 top- k 采样 ($k = 50$) 和 top- p 采样 ($p = 0.9$)，并将温度参数设置为 0.7，在保证结果可靠的同时促进生成结果的多样性。

- **LLaMA**. LLaMA 根据具体任务的不同采用了多样化的解码策略。例如，在问答任务中使用了贪心搜索策略；而在代码生成任务中，分别采用了温度设置为 0.1（针对 pass@1）和 0.8（针对 pass@100）的采样策略。

- **OpenAI API**. OpenAI 支持多种基本的解码策略，包括贪心搜索（通过将温度

参数 `temperature` 设置为 0 实现)、束搜索 (通过 `best_of` 参数设置)、温度采样 (通过调整 `temperature` 参数) 以及 Top- p 采样 (通过 `top_p` 参数)。此外, 它也支持基于出现惩罚和频率惩罚的重复惩罚机制, 分别通过 `presence_penalty` 和 `frequency_penalty` 来控制。

用户输入: 人类的正常体温大约是多少摄氏度?

贪心搜索的回复

模型输出: 人类正常体表温度大约在 37 摄氏度 (98.6 华氏度) 左右。然而, 体温可能因个体差异、环境条件以及测量方式的不同而有所变化。正常情况下, 人体内部的核心温度保持相对稳定, 而体表温度则会因外部环境和身体活动等元素而有所波动。因此, 37 摄氏度通常被视为人类正常体温的参考值, 但实际体温可能会有轻微的变化。

贪心搜索和频率惩罚 ($\alpha = 1$) 的回复

模型输出: 人类正常体表温度大约在 37 摄氏度 (98.6 华氏度) 左右。然而, 体温可能因个体差异、环境条件以及测量方式的不同而有所变化。由于人类体内有调节体温的机制, 因此正常情况下人体内部温度保持相对稳定, 不会出现剧烈波动。

Top- p 采样 ($p = 0.95$) 的回复

模型输出: 人类正常体表温度大约在 36.5 到 37.5 摄氏度之间。然而, 人体内部正常腋下体温通常在 36.5 到 37.2 摄氏度之间, 而直肠温度则通常比腋下体温略高 0.2 到 0.4 摄氏度。健康成人的正常体表温度通常是指皮肤表面的温度, 而正常腋下温度则反映了体内部分的温度变化。在医学检查中, 腋下体温通常被认为比口腔和直肠体温更准确, 因为体温测量方式不同。

例 9.1 不同解码策略时 YuLan 模型的回复

9.2 解码加速算法

在上述内容已经介绍了多种解码算法。然而在实现过程中, 由于自回归算法的序列化生成特点, 使得解码算法存在效率较低的问题。在本节中, 我们将首先定量分析解码算法的效率, 再介绍针对性的效率优化算法。

9.2.1 解码效率分析

本节将首先介绍解码的两个阶段（即全量解码和增量解码），然后对这两个阶段分别进行效率分析，从而发现解码过程中的效率瓶颈所在。以下分析基于现有工作 [239]。

算法 3 基于键值缓存优化的贪心解码

输入：模型 $\mathcal{M}$ ，输入词元序列 u

输出：输出词元序列 y

```

1:  $P, K_{past}, V_{past} = \mathcal{M}(u)$ 
2:  $u' = \arg \max P$ 
3:  $u \leftarrow u \oplus [u']$ 
4: while  $u'$  不是结束词元且  $u$  的长度不超过预设长度 do
5:    $P, K, V = \mathcal{M}(u', K_{past}, V_{past})$ 
6:    $u' = \arg \max P$ 
7:    $u \leftarrow u \oplus [u']$ 
8:    $K_{past}, V_{past} \leftarrow K_{past} \oplus K, V_{past} \oplus V$ 
9: end while
10:  $y \leftarrow u$ 

```

全量解码与增量解码

在第 9.1.1 节中我们已经展示了原始的自回归解码流程，如算法 2 所示。具体来说，观察循环内相邻的两次前向传播过程，需要进行 $P = \mathcal{M}(u)$ 和 $P' = \mathcal{M}(u \oplus [u'])$ 两次计算。根据公式 5.2 所示，这两次前向传播的具体计算操作为： $Q = XW^Q$ 和 $Q' = [X \oplus [u']]W^Q = XW^Q \oplus [u']W^Q$ ，可以发现后者相比前者只需要额外计算一次新生成词元 u' 相关的状态即可。同样的，对于公式 5.3、公式 5.4 和公式 5.8 等公式，都是只需要多计算新生成词元 u' 相关的状态即可，而不需要重复计算之前词元的状态。对于注意力计算的公式 5.7 则稍有不同，其公式为：

$$\text{Attention}(Q', K', V') = \text{softmax}\left(\frac{QK'^T}{\sqrt{D}}\right)V' \oplus \text{softmax}\left(\frac{(u'W^Q)K'^T}{\sqrt{D}}\right)V', \quad (9.4)$$

其中， $K' = K \oplus (u'W^K)$ ， $V' = V \oplus (u'W^V)$ 。直观来说，对于新生成词元 u' 而言，其需要作为查询与之前词元的键和值进行注意力计算。因此，可以通过缓存之前序列的键值状态，每次生成下一个词元时利用缓存的键值矩阵计算当前的多头注意力，这称为键值缓存（Key-Value Caching）优化。

算法 3 展示了基于键值缓存优化的贪心解码的伪代码。总体来说，解码算法主要可以分为两个阶段：（1）全量解码阶段，对于输入序列，一次性地计算其状态并缓存键值矩阵（算法 3 第 1 至 3 行）；（2）增量解码阶段，只计算上一步新生

成词元的状态，并不断地以自回归方式生成新词元并对应更新键值缓存，直到生成结束（算法 3 第 4-9 行）。

解码效率的定量评估指标

为了定量地进行解码效率的分析，我们首先引入 GPU 算力和 GPU 带宽这两个概念，以此来评估特定 GPU 的性能。算力是指 GPU 每秒能够进行的浮点运算次数，单位是 FLOP/s；带宽是该显卡每秒能够进行的显存读写量，单位是 byte/s。算力和带宽的比值被称为该 GPU 的计算强度上限 I_{max} ，单位为 FLOP/byte。以 A100 GPU 为例，该显卡半精度浮点数的算力为 312 TFLOP/s，即每秒能进行 3.12×10^{14} 次半精度浮点数运算；同时，所对应的带宽为 2039 GB/s。通过按照上述定义进行计算，可以获得 A100 GPU 的计算强度上限约为 142.51 FLOP/byte。

同样地，模型在运行时也有相应的两个性能指标为：运算量和访存量。运算量是指运行该模型需要的总浮点计算数，单位为 FLOP；访存量是运行该模型的过程中所需的显存读写量，单位为 byte。与 GPU 的计算强度相似，运算量和访存量的比值被称为该模型的计算强度 I ，单位为 FLOP/byte。当模型的计算强度 I 小于 GPU 的计算强度上限 I_{max} 时，这说明 GPU 的理论最高显存读写速度低于实际运算所需速度，因此模型实际的运行效率将主要受到显存读写速度的影响，这种情况称为带宽瓶颈；反之，当 I 大于 I_{max} 时，说明 GPU 的理论最高浮点运算速度低于实际运算所需速度，因此模型的运行效率将主要受到算力的影响，这种情况称为计算瓶颈。

表 9.1 全量解码的运算量、访存量和计算强度（表格来源：[239]）

计算公式	运算量	访存量	计算强度
① $\mathbf{Q}, \mathbf{K}, \mathbf{V} = \mathbf{XW}^{\mathbf{Q}, \mathbf{K}, \mathbf{V}}$	$6BTH^2$	$O(BTH + H^2)$	$O\left(\frac{1}{\frac{1}{H} + \frac{1}{BT}}\right)$
② $\mathbf{Q}, \mathbf{K} = \text{RoPE}(\mathbf{Q}, \mathbf{K})$	$6BTH$	$O(BTH)$	$O(1)$
③ $\mathbf{O} = \text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$	$4BT^2ND + 4BT^2N$	$O(BT^2N + BTND)$	$O\left(\frac{1 + \frac{1}{D}}{\frac{1}{D} + \frac{1}{T}}\right)$
④ $\mathbf{X} = \mathbf{OW}^{\mathbf{O}}$	$2BTH^2$	$O(BTH + H^2)$	$O\left(\frac{1}{\frac{1}{H} + \frac{1}{BT}}\right)$
⑤ $\mathbf{X} = \text{Add\&Norm}(\mathbf{X})$	$5BTH$	$O(BTH + H)$	$O\left(\frac{1}{1 + \frac{1}{BT}}\right)$
⑥ $\mathbf{G}, \mathbf{U} = \mathbf{X}[\mathbf{W}^{\mathbf{G}}, \mathbf{W}^{\mathbf{U}}]$	$4BTHH'$	$O(BTH + BTH' + HH')$	$O\left(\frac{1}{\frac{1}{H} + \frac{1}{H'} + \frac{1}{BT}}\right)$
⑦ $\mathbf{D} = \text{SiLU}(\mathbf{G}) \cdot \mathbf{U}$	$2BTH'$	$O(BTH')$	$O(1)$
⑧ $\mathbf{X} = \mathbf{DW}^{\mathbf{D}}$	$2BTHH'$	$O(BTH + BTH' + HH')$	$O\left(\frac{1}{\frac{1}{H} + \frac{1}{H'} + \frac{1}{BT}}\right)$
⑨ $\mathbf{X} = \text{Add\&Norm}(\mathbf{X})$	$5BTH$	$O(BTH + H)$	$O\left(\frac{1}{1 + \frac{1}{BT}}\right)$

运算量、访存量和计算强度估计

本部分将以 LLaMA 模型为例，分析在全量解码和增量解码过程中的运算量和访存量¹并得到相应的计算强度。全量解码运算量与训练阶段前向传播的运算量相同，表 9.1 中列举了全量解码阶段的运算量，具体分析过程详见第 6.4.2 节。

小贴士 (矩阵乘法的读写量)

矩阵 $A \in \mathbb{R}^{n \times m}$ 和矩阵 $B \in \mathbb{R}^{m \times p}$ 相乘所需的读写量为 $O(nm + mp + np)$ 。

接下来分析全量解码阶段每个部分的访存量（符号设置与第 6.4.2 节中相同）：

- 自注意力部分. 自注意力机制的查询矩阵需要进行线性映射（表 9.1 中公式①），其中 $X \in \mathbb{R}^{B \times T \times H}$ ， $W^Q \in \mathbb{R}^{H \times H}$ ，因此该操作的访存量为 $O(BTH + H^2)$ ，键和值的线性映射同理也是该复杂度。而多头注意力的计算部分（公式③），访存量共分为三部分：首先，计算查询 Q 和键 K 的矩阵乘法（ $Q, K \in \mathbb{R}^{B \times N \times T \times D}$ ），访存量为 $O(BTND + BT^2N)$ ；然后，将 Q 和 K 相乘后的矩阵进行标准化操作除以 $\sqrt{D}$ ，再计算 softmax，这一过程的访存量为 $O(BT^2N)$ ；最后，将上述得到的矩阵与值 V 做矩阵乘法，访存量为 $O(BT^2N + BTND)$ 。综合上述三个部分，多头注意力访存量为 $O(BT^2N + BTND)$ 。当注意力计算结束后，公式④输出的线性变换与公式①类似，其访存量为 $O(BTH + H^2)$ 。

- 前馈网络部分. 前馈网络中有三个线性变换运算（公式⑥和⑧），其中 $X \in \mathbb{R}^{B \times T \times H}$ ， $W^G \in \mathbb{R}^{H \times H'}$ ，因此其访存量为 $O(BTH + BTH' + HH')$ ；而公式⑦中的激活函数计算，访存量为 $O(BTH')$ 。

- 其他部分. 最后，LLaMA 模型中的 RoPE（公式②）、归一化和残差连接（公式⑤和⑨）的访存量都是 $O(BTH)$ 。

接下来分析增量解码，其运算量、访存量和计算强度详见表 9.2。与全量解码相比，增量解码在大部分运算上只需要将输入词元长度视为 $T = 1$ 即可，唯一不同的地方是多头注意力计算部分（公式 9.4）和额外的键值缓存更新操作（算法 3 第 8 行）。因此除了这两个操作外，增量解码其他部分的运算量和访存量可以通过将全量解码公式中的 T 替换为 1 来获得。对于增量解码阶段的多头注意力计算，由于键和值依然是矩阵形式，所以其访存量变为 $O(BTN + BTND + BND)$ 。而键值缓存的更新操作，在不使用显存优化算法的情况下访存量为 $O(BTND)$ ，如果使用了例如 PagedAttention（见第 9.2.2 节）等优化技术，其访存量可以降低为 $O(BND)$ 。

¹在访存量的分析中，本书使用渐进上界符号 O 来表示访存量的复杂度，而复杂度的常数部分与 GPU 实际的算法相关。

表 9.2 增量解码的运算量、访存量和计算强度（表格来源：[239]）

	运算量	访存量	计算强度
① $q, k, v = XW^{QKV}$	$6BH^2$	$O(BH + H^2)$	$O\left(\frac{1}{\frac{1}{H} + \frac{1}{B}}\right)$
② $q, k = \text{RoPE}(q, k)$	$6BH$	$O(BH)$	$O(1)$
③ $K, V = \text{Cache}(k, v)$	-	$O(BTND)$ 或 $O(BND)$	-
④ $o = \text{Attn}(q, K, V)$	$4BTND + 4BTN$	$O(BTN + BTND + BND)$	$O\left(\frac{1 + \frac{1}{D}}{1 + \frac{1}{D} + \frac{1}{T}}\right)$
⑤ $X = oW^O$	$2BH^2$	$O(BH + H^2)$	$O\left(\frac{1}{\frac{1}{H} + \frac{1}{B}}\right)$
⑥ $X = \text{Add\&Norm}(X)$	$5BH$	$O(BH + H)$	$O\left(\frac{1}{1 + \frac{1}{B}}\right)$
⑦ $g, u = X[W^G, W^U]$	$4BHH'$	$O(BH + BH' + HH')$	$O\left(\frac{1}{\frac{1}{H} + \frac{1}{H'} + \frac{1}{B}}\right)$
⑧ $d = \text{SiLU}(g) \cdot u$	$2BH'$	$O(BH')$	$O(1)$
⑨ $X = dW^D$	$2BHH'$	$O(BH + BH' + HH')$	$O\left(\frac{1}{\frac{1}{H} + \frac{1}{H'} + \frac{1}{B}}\right)$
⑩ $X = \text{Add\&Norm}(X)$	$5BH$	$O(BH + H)$	$O\left(\frac{1}{1 + \frac{1}{B}}\right)$

内存墙及瓶颈分析

基于上述分析，可以获得全量解码和增量解码阶段的运算量和访存量，进而能够通过计算其比值得到每个操作的计算强度 I 和计算强度上限 I_{max} ，从而更好地发现解码过程中的瓶颈操作。

这里，将以 LLaMA (7B) 模型为例进行说明（其中 $N = 32, D = 128, H = 4096$ ）。在全量解码阶段，假设批次大小为 8，序列长度为 1024（即 $B = 8, T = 1024$ ），将这些具体数值代入到表 9.1 的计算强度的公式中可以计算得到，各个线性变换（公式①④⑥⑧）的计算强度大约为 2730.67，多头注意力（公式③）的计算强度大约为 114.67，而其余操作（公式②⑤⑦⑩）的计算强度都在 1 左右。在使用 A100 (80G) 的 GPU 时（计算强度上限 I_{max} 为 142.51），各个线性变换和多头注意力部分的计算强度都高于或接近这个值，考虑到这些操作占据了全量解码的绝大多数运算，可以说全量解码阶段是受限于 GPU 浮点数计算能力的（即计算瓶颈）。

类似地，将这些数值代入到表 9.2 中的计算强度公式，可以得到在增量解码阶段各个线性变换和多头注意力的计算强度都不超过 8，远小于 A100 GPU 的计算强度上限 142.51，这表明增量解码阶段是受限于 GPU 显存读写速度的（即显存瓶颈），这种问题通常被称为内存墙（Memory Wall）问题。基于上述分析，可以看到解码阶段的低效问题主要出现在增量解码阶段，接下来将从系统优化和解码策略优化两个维度来介绍增量解码阶段的改进方法。

9.2.2 系统级优化

针对“内存墙”问题，一个直观的解决方案是减少相关操作的访存量，从而达到提升计算强度的目的。本节将介绍一些系统级优化方法来实现减少访存量的目的。

FlashAttention

FlashAttention [240] 是一种针对原始注意力模块（公式 5.5）的优化方案，可以大幅减少注意力计算中的访存量，从而提升计算强度。它的核心思路是尽可能减少对于中间结果的保存，进而直接得到最终结果。根据注意力的计算方法 $\text{softmax}(\frac{QK^T}{\sqrt{D}})V$ ，可以发现其中需要保留多个中间结果，如 QK^T 和 softmax 后的注意力分布矩阵。这些中间结果需要频繁写入显存，因此导致了大量的显存读写操作。而 FlashAttention 通过矩阵分块和算子融合等方法，将中间结果一直保留在缓存中，直到获得最终结果后再写回显存中，从而减少了显存读写量。

FlashAttention 有效地减少了访存量，同时也降低了峰值显存的占用量（第 6.4.4 节）。例如，使用了 FlashAttention 的 LLaMA-2 (7B) 在序列长度为 2,048、批次大小为 8 的情况下，注意力操作的时间仅需传统注意力的十分之一。

PagedAttention

PagedAttention [174] 是针对键值缓存拼接和注意力计算的优化操作，能够有效降低这两个运算部分的访存量，从而提高计算效率。在键值缓存拼接操作中，传统的实现方法会在每次拼接时新分配一段显存空间，然后拷贝原始键值缓存和新生成词元的状态到新分配的显存中去，这容易导致反复的显存读写，并且产生较多的显存碎片。为此，PagedAttention 引入了操作系统中显存分页管理的方法，预先将显存划分成若干块给之后的键值缓存“预留空间”，从而显著减少了拼接时反复分配显存的操作。

此外，PagedAttention 还优化了注意力计算操作，提高了计算的并行度从而减少其访存量。具体来说，增量解码阶段是以当前词元作为查询向量，与之前序列的键值缓存进行注意力计算的过程。考虑到键值缓存通常会较长，PagedAttention 采用了上述的分页管理操作，并使用算子融合的方法将查询向量与多个分页的键值缓存并行地进行计算，从而提升了计算效率。

批次管理优化

在传统的解码操作中，通常会等待一整个批次的所有实例都结束后再进行下

一个批次的计算。然而，一个批次内的不同实例往往生成长度各异，因此经常会出现等待某一条实例（输出长度最长的实例）生成的情况。从表 9.2 中可以发现，在计算批次大小 B 较小时，计算强度很低，因此解码效率低下。

为了解决这个问题，批次管理优化旨在通过增加计算中的批次大小来提高计算强度。一个代表性的方法是 vLLM（细节参考第 9.2.4 节）所提出的连续批处理（Continuous Batching）技术 [174]。该技术不同于传统确定顺序的定长批次处理方式，而是将每个输入实例视为一个请求，每个请求的处理过程可以分解为全量解码阶段和若干个单步增量解码阶段。在实现中，连续批处理技术会通过启发式算法来选择部分请求进行全量解码操作，或者选择一些请求进行单步增量解码操作。通过这样细粒度的拆分，连续批处理技术在一步操作中能够容纳更多的请求（相当于提高批次大小），从而提升了计算强度。进一步，DeepSpeed-MII（细节参考第 9.2.4 节）提出了动态分割技术，将全量解码部分进一步拆分为多个子操作，其可以在一次计算中选择一些请求同时进行全量解码和增量解码操作，进而获得更大的批次和更高的解码吞吐量。

通过批次管理优化技术，模型可以更好地适配线上大模型应用服务。例如，ChatGPT 的网页服务端通常会面临着大规模的用户请求，线上部署模型需要尽快地将生成结果返回给用户。传统的批次生成方法会带来很高的服务时延（新请求必须等待前一个完成），而批次管理技术可以细粒度地分割不同请求的处理阶段，使得不同请求的处理过程可以同时进行处理，从而实现更为高效的线上服务。

9.2.3 解码策略优化

除了直接解决系统级别的内存墙问题，许多研究工作提出了针对自回归解码策略的改进方法，从而提高解码效率。下面主要介绍四种解码优化算法，包括推测解码（Speculative Decoding）、非自回归解码（Non-autoregressive Decoding）、早退机制（Early Exiting）与级联解码（Cascade Inference）。

推测解码

考虑到语言建模的生成步骤有难易之分。例如，预测“世界最高峰是”的下一个词可能较为困难，但是预测“世界最高峰是珠”的下一个词可能相对简单，即使是小模型也可能成功预测。基于这个想法，推测解码 [241] 提出首先使用相对较小但解码更为高效的模型（例如 n 元统计模型或者小型预训练模型）自回归地生成若干个词元，然后再由大模型对这个片段进行一次验证（大模型一次验证与一

次解码的时间消耗相当), 来判断是否每个词元都是当前步骤概率最高的输出, 随后大小模型持续迭代此过程直到解码结束。推测解码不会降低大模型解码的质量, 实验测试表明能够带来约两倍左右的解码加速, 是目前使用较多的解码策略优化方案。

为了方便读者理解, 这里我们构造一个例子来说明推测解码的工作流程。假设输入为“我与父亲不相见已二年余了, 我最不能”, 大模型的输出为“忘记的是他的背影”, 总共要生成 8 个单词 (为了讲解方便以单词为基础单位), 需要 8 次迭代的生成过程。如果使用推测解码, 第一步先使用小模型生成 3 个单词, 假设结果为“忘记他”, 然后使用大模型进行验证, 由于前两个词正确第三个词错误, 则同时将其修正为“忘记的”; 第二步, 再使用小模型生成 3 个单词, 假设结果为“日子是”, 大模型验证第一个词便发现错误, 同时将其修正为“他”; 第三步, 小模型生成“的背影”, 大模型验证均正确, 生成结束。回顾上述生成过程, 小模型共计生成了 9 次, 大模型验证了 3 次 (验证同时可以修正错误), 相较于大模型的 8 次生成过程可以有一定的效率提升。

级联解码

与推测解码有类似的想法, 级联解码考虑到不同请求的难易度不同, 分别使用不同规模的模型来处理请求, 从而实现最小化解码时间的效果。FrugalGPT [242] 引入了一系列模型 (按照效率从高到低), 然后将一个请求依次给这些模型进行生成, 随后引入了一个专门训练的二分类模型来判断模型的生成结果是否符合任务要求。如果结果可靠就不需要再给之后的模型进行生成, 以实现提升解码效率的目的。该策略也可以应用于不同的开源模型、商业 API 等, 用户可以根据自己的需求设定分类器的判断阈值, 从而平衡生成效率与生成质量。

非自回归解码

现有解码策略普遍采用自回归的解码机制, 即逐个词元进行生成, 这是导致解码效率低下的重要原因。因此, 机器翻译领域的研究人员提出了非自回归的解码机制, 可以基于输入并行地一次性生成所有的词元。但是这种方法的生成质量往往与自回归方法有一定差距, 因此有研究工作尝试结合这两种方法, 进一步提出了半自回归解码, 每一次生成一组词元 (例如 3 至 10 个词元), 再以这些词元作为输入继续生成下一组。然而, 现有的大模型都是预测下一个词进行预训练的, 无法直接进行非 (半) 自回归生成。为了解决这个问题, Medusa [243] 在 Vicuna 模型的基础上, 额外训练了两个预测头来分别预测第二个词和第三个词, 因此可以

达到一次生成三个词元的效果。但需注意的是，尽管这些非（半）自回归策略在效率上有所提升，但仍然不能达到自回归解码的效果。因此其很少单独使用，通常可以用于推测解码中的候选片段生成，进而加速大模型的解码流程。例如 Medusa 预测片段之后，需要原始 Vicuna 模型进行验证来保证生成质量。

早退机制

有研究工作发现，在多层 Transformer 模型中，可能不需要经过所有层的计算，模型就可以较为可靠地预测下一个词的生成。基于这种想法，研究人员提出了基于早退机制的生成方式。在模型的解码过程中，可以通过设置相应的早退判断条件。当早退条件满足时结束网络层的前向传递，直接生成相关的词元，从而提升解码效率。早期一种常见的早退判断方法是对于 Transformer 每一层的输出都使用预测头得到在词表上的概率分布，然后计算该分布的熵。如果熵值小于预定义的阈值（即某一个词的概率显著较高），则可以判断为早退，不进行后续层的计算。在实现中，可以通过调整该阈值来平衡解码效率与生成质量。最近的研究工作提出了混合深度方法 [244]，来动态调整每一层的计算量。类似于 MoE 网络（详见第 5.2.6 节），混合深度方法对于每一层的输入通过路由网络计算得分，如果该得分高于预先设定的阈值则进行该层的后续计算，否则直接跳过该层的计算。与传统早退机制直接跳过后续所有层计算相比，混合深度方法有选择性的跳过了部分层，因此可以更好地利用模型中不同层的特性。

9.2.4 解码代码实践

在本节中，我们将介绍一些常用于大模型解码的代码库，并探讨它们针对解码过程所进行的关键优化策略。随后，将着重介绍 vLLM 代码库，并通过具体代码示例来详细演示其使用方式。

常见代码库介绍

目前的代码库主要围绕上一节中提到的系统级优化进行，它们兼容大部分的开源模型（例如 LLaMA），在实际部署中可以方便应用。下面主要介绍一下几个常用于大模型解码的代码库。

- *llama.cpp*. llama.cpp 是一款完全基于 C/C++ 实现的代码库，具有较好的跨平台兼容性，能够在多种计算设备（如 CPU、英伟达 GPU、AMD GPU 以及苹果芯片）上运行。此外，llama.cpp 还支持多种量化精度，范围从 1.5 比特到 8 比特不

等，能够显著降低显存消耗。

- **vLLM**. vLLM 是一个快速、高效且便捷的代码库，它专门针对解码效率进行了大量优化。vLLM 通过对键值缓存进行分页存储，并结合 PagedAttention 技术，显著提升了解码时注意力的计算效率。同时它还支持多种解码策略，引入了批次管理优化技术，能够很好地在真实场景中部署大模型。此外，vLLM 有较强的兼容性，可以与大量 Transformers 库中的开源模型无缝集成。

- **DeepSpeed-MII**. DeepSpeed-MII 代码库是由微软开发的一个用于大语言模型解码的代码库。它能够支持多种解码策略，并实现了批次管理优化和张量并行解码等技术。为了充分挖掘 GPU 的计算潜能，DeepSpeed-MII 引入了动态分割技术，通过将输入提示拆分为多个子块，并将全量解码和增量解码的请求有机融合，进而实现了批次数据的增加和解码吞吐量的提升。

- **FlexFlow**. FlexFlow 代码库针对推测解码算法进行了优化，进一步提升了推测解码的效率。在早期的推测解码算法中，小模型解码和大模型验证的过程是交替进行的，并且一次只能验证一个推测。FlexFlow 设计了树形注意力机制，该机制将小模型的多个推测结果拼接到一条序列中，通过修改注意力掩码让大模型实现一次验证多个推测的效果，有效提升了计算并行度。

vLLM 代码实践

下面以 vLLM 为例，展示如何使用 meta-llama/Llama-2-7b-chat-hf 模型进行解码生成。

```

1  import vllm
2
3  # 符合 LLaMA-2 Chat 格式的三个提示
4  prompts = [
5      '[INST] How are you? [/INST]',
6      '[INST] 1 + 1 = ? [/INST]',
7      '[INST] Can you tell me a joke? [/INST]',
8  ]
9
10 # 初始化 vLLM 的模型
11 model = vllm.LLM(model='meta-llama/Llama-2-7b-chat-hf')
12
13 # 设置 vLLM 的解码参数
14 sampling_params = vllm.SamplingParams(
15     temperature=0, # 温度设置为 0 表示贪心搜索
16     max_tokens=2048, # 新生成 token 数上限
17     presence_penalty=0, # 存在惩罚系数
18     frequency_penalty=0, # 频率惩罚系数
19 )
20
21 # 调用 vLLM 的模型进行生成

```

```

22 out = model.generate(prompts, sampling_params=sampling_params)
23 for prompt, it in zip(prompts, out):
24     print(f'input = {prompt!r}\noutput = {it.outputs[0].text!r}')
25
26 # 样例 1
27 # input = '[INST] How are you? [/INST]'
28 # output = ' I\'m just an AI, I don\'t have feelings or emotions like humans
↪ do, so I don\'t have a physical state of being such as "good" or "bad."
↪ I\'m here to help answer your questions and provide information to the
↪ best of my ability, so please feel free to ask me anything!'
29
30 # 样例 2
31 # input = '[INST] 1 + 1 = ? [/INST]'
32 # output = ' The answer to 1 + 1 is 2.'
33
34 # 样例 3
35 # input = '[INST] Can you tell me a joke? [/INST]'
36 # output = " Of course! Here's a classic one:\n\nWhy don't scientists trust
↪ atoms?\n\nBecause they make up everything!\n\nI hope that made you
↪ smile! Do you want to hear another one?"

```

此外，vLLM 也可以像 ChatGPT 的网页端一样启动网络服务功能，将模型一直挂载运行，相应的启动命令如下所示：

```

1 python -m vllm.entrypoints.api_server --port 8000 --model
↪ meta-llama/Llama-2-7b-chat-hf --dtype bfloat16 --tensor-parallel-size 1

```

其中，port 参数用于设定端口，tensor-parallel-size 是张量并行数，主要用于多卡部署，通常设置为卡数。进一步，可以通过以下指令访问这个端口，请求参数和 OpenAI API 基本一致：

```

1 curl http://localhost:8000/v1/completions \
2     -H "Content-Type: application/json" \
3     -d '{
4         "prompt": "[INST] 1 + 1 = ? [/INST]",
5         "max_tokens": 20,
6         "temperature": 0
7     }'

```

9.3 低资源部署策略

由于大模型的参数量巨大，在解码阶段需要占用大量的显存资源，因而在实际应用中的部署代价非常高。在本章中，我们将介绍一种常用的模型压缩方法，即

模型量化 (Model Quantization)，来减少大模型的显存占用，从而使得能够在资源有限的环境下使用大模型。

9.3.1 量化基础知识

在神经网络压缩中，量化通常是指从浮点数到整数的映射过程 [245]，目前比较常用的是 8 比特整数量化，即 *INT8* 量化。针对神经网络模型，通常有两种类型的数据需要进行量化，分别为 权重量化（也称为模型参数量化）和激活（值）量化，它们都以浮点数形式进行表示与存储。

量化的数学表述

量化的过程可以表示为一个函数，该函数将连续的输入映射到离散的输出集合。一般来说，这个过程涉及到四舍五入或截断等近似操作。下面介绍一个一般形式的量化函数：

$$\mathbf{x}_q = R(\mathbf{x}/S) - Z. \quad (9.5)$$

通过上述数学变换，量化算法将浮点数向量 $\mathbf{x}$ 转化为量化值 $\mathbf{x}_q$ 。其中， S 表示缩放因子，用于确定裁剪范围， Z 表示零点因子，用于确定对称或非对称量化， $R(\cdot)$ 表示将缩放后的浮点值映射为近似整数的取整操作。一般来说，裁剪范围对于量化性能有很大影响，通常需要根据实际数据分布进行校准，可以通过静态（离线）或动态（运行时）方式。

作为上述变换的逆过程，反量化（Dequantization）对应地从量化值中恢复原始值，该过程首先加上零点因子，然后乘以缩放因子：

$$\tilde{\mathbf{x}} = S \cdot (\mathbf{x}_q + Z). \quad (9.6)$$

进一步，可以定义量化误差是原始值 $\mathbf{x}$ 和恢复值 $\tilde{\mathbf{x}}$ 之间的数值差异： $\Delta = \|\mathbf{x} - \tilde{\mathbf{x}}\|_2^2$ 。

量化的策略

基于上述量化函数的定义形式，下面介绍一些对于量化函数常见的分类与实现策略。

- 均匀量化和非均匀量化. 根据映射函数的数值范围是否均匀，可以将量化分为两类：均匀量化和非均匀量化。均匀量化是指在量化过程中，量化函数产生的量化值之间的间距是均匀分布的。这种方法通常用于将连续的数据转换为离散的表示，以便在计算机中进行高效处理。与此不同，在非均匀量化方法中，它的量

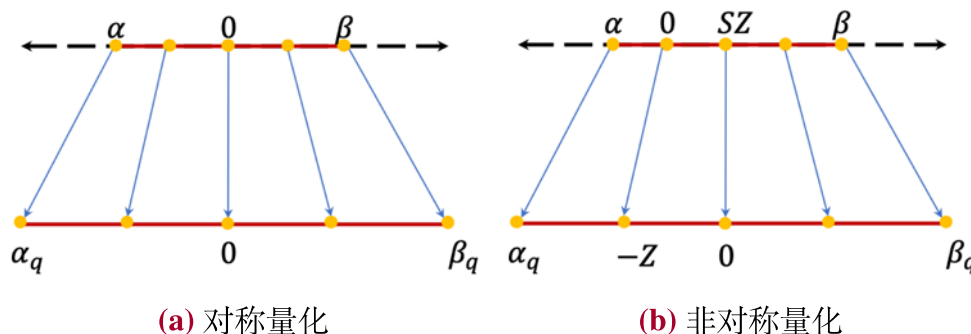


图 9.6 对称量化和非对称量化对比

化值不一定均匀分布，可以根据输入数据的分布范围而进行调整。其中，均匀量化方法因其简单和高效的特点而在实际中被广泛使用。

- 对称量化和非对称量化. 根据零点因子 Z （公式 9.5）是否为零，均匀量化可以进一步分为两类：对称量化（ $Z = 0$ ）和非对称量化（ $Z \neq 0$ ）。对称量化与非对称量化的一个关键区别在于整数区间零点的映射，对称量化需要确保原始输入数据中的零点（ $x = 0$ ）在量化后仍然对应到整数区间的零点。而非对称量化则不同，根据前面的公式可以看出此时整数区间的零点对应到输入数值的 $S \cdot Z$ 。为了方便讨论，这里以一个常见的 8 比特量化为例进行介绍。如图 9.6 (a) 和图 9.6 (b) 所示，对称量化将输入数值 x 通过一个映射公式转换为八比特整数的表示范围内，如果是有符号整数，则该范围可以设置为 $[-128, 127]$ ，适用于 x 的数值大致分布在零点两侧的情况，如果是无符号整数，则设置为 $[0, 255]$ ，适用于输入数值基本都分布在零点一侧的情况。

- 量化粒度的选择. 量化算法通常针对一个批次的数据进行处理，其中批次的规模大小就反应了量化粒度，可以由算法设计人员进行选择。在神经网络模型中，输入数据和模型权重通常是以张量的形式进行表示与组织的。首先，如果每个张量只定义一组量化参数（即 S 和 Z ），这称为按张量量化。为了提高量化的精度，可以针对每个张量定义多组量化参数，例如可以为权重矩阵的列维度（也称为“通道”）设置特定的量化参数，称为按通道量化。还有一些研究工作采用了更细粒度的量化方案，对一个通道的数值细分为多个组，即按组的方式进行量化。在神经网络量化中，从按张量到按组，量化粒度越来越小，且使用较小的粒度通常可以提高量化的准确性，有效保持原始模型的性能。但是由于引入了更多的量化参数，在使用时会带来额外的计算开销。相反，按张量量化的粒度较粗，可能会引入较大的误差，但由于在硬件实现上更加简单，也是一种常见的量化粒度选

择策略。在实践中，量化粒度需要根据具体任务和模型进行选择，应该采用可以平衡量化准确性以及额外计算开销的合适粒度。

量化方法示例与实践

为了帮助读者更好地理解量化算法，本部分内容将通过一个具体的例子对于量化算法进行介绍，然后给出对应的实现算法。

- 非对称量化示例. 为了方便介绍，这里仍然以 8 比特量化为例。给定输入数据 $\mathbf{x} = [[1.2, 2.4, 3.6], [11.2, 12.4, 13.6]]$ 。首先实现一种非对称的均匀量化，根据输入数据的范围可知 $\mathbf{x} \in [1.2, 13.6]$ ，我们希望将其量化到整数表示范围 $[-128, 127]$ 。为此，我们需要设置对应的量化参数 S 和 Z ，以确保 1.2 和 13.6 被映射到 -128 和 127。根据公式 9.6，可以得到下面的二元一次方程组：

$$S \cdot (127 + Z) = 13.6, \quad (9.7)$$

$$S \cdot (-128 + Z) = 1.2. \quad (9.8)$$

需要注意的是，这里 S 是浮点数而 Z 是整数，计算可得 $S = \frac{13.6-1.2}{127-(-128)} = 0.0486$ ， $Z = -152$ 。根据这两个量化参数，结合公式 9.5，可以得到量化后的数值为 $\mathbf{x}_q = [[-127, -103, -78], [78, 103, 127]]$ 。反量化后的数据为 $\tilde{\mathbf{x}} = [[1.2157, 2.3827, 3.5984], [11.1843, 12.4000, 13.5671]]$ 。

- 对称量化示例. 对于对称的均匀量化来说，数据中的零点要求被映射到对应整数区间到零点，此时参数 $Z = 0$ ，如图 9.6 (b) 所示。仍然以 8 比特量化为例，为了保证所有的数据都可以映射到对应的整数区间，对称量化需要确保整数区间覆盖到的输入数据 $\mathbf{x}$ 的取值范围是 $[-13.6, 13.6]$ 。此时 $S = 0.1067$ ，量化后得到 $\mathbf{x}_q = [[11, 22, 34], [105, 116, 127]]$ 。反量化后的数据为 $\tilde{\mathbf{x}} = [[1.1733, 2.3467, 3.6267], [11.2000, 12.3733, 13.5467]]$ 。相较于非对称量化覆盖到的数据取值范围 $[1.2, 13.6]$ ，可以看出对称量化覆盖的范围更大。但是，由于对称量化的整型数据表示范围超出了实际数据的数值区间 $[1.2, 13.6]$ 。特别是在区间 $[-13.6, 1.2)$ 范围内，没有实际数值存在，这会导致大量整型数值的浪费。此外，由于覆盖的范围更大，对称量化会引入更大的量化误差。可以看到，对称量化的数据反量化后的结果与原始结果的差异要大于非对称量化。

- 代码实践. 接下来通过代码演示来了解量化过程以及反量化过程。首先，这里定义了“quantize_func”和“dequantize_func”两个函数，分别代表量化和反量化函数。然后，根据前面非对称量化示例中所介绍的方法，针对输入进行量

化参数的计算，并基于这些参数对输入进行量化和反量化处理。具体代码如下所示：

```

1 import torch
2 import numpy as np
3
4 def quantize_func(x, scales, zero_point, n_bits=8):
5     x_q = (x.div(scales) + zero_point).round()
6     x_q_clipped = torch.clamp(x_q, min=alpha_q, max=beta_q)
7     return x_q_clipped
8
9 def dequantize_func(x_q, scales, zero_point):
10    x_q = x_q.to(torch.int32)
11    x = scales * (x_q - zero_point)
12    x = x.to(torch.float32)
13    return x
14 if __name__ == "__main__":
15     # 输入配置
16     random_seed = 0
17     np.random.seed(random_seed)
18     m = 2
19     p = 3
20     alpha = -100.0 # 输入最小值为-100
21     beta = 80.0 # 输入的最大值为 80
22     X = np.random.uniform(low=alpha, high=beta,
23                           size=(m, p)).astype(np.float32)
24     float_x = torch.from_numpy(X)
25     # 量化参数配置
26     num_bits = 8
27     alpha_q = -2**(num_bits - 1)
28     beta_q = 2**(num_bits - 1) - 1
29     # 计算 scales 和 zero_point
30     S = (beta - alpha) / (beta_q - alpha_q)
31     Z = int((beta * alpha_q - alpha * beta_q) / (beta - alpha))
32     # 量化过程
33     x_q_clip = quantize_func(float_x, S, Z)
34     print(f" 输入: \n{float_x}\n")
35     # tensor([[ -1.2136,  28.7341,   8.4974],
36     #         [-1.9210, -23.7421,  16.2609]])
37     print(f"{num_bits}比特量化后: \n{x_q_clip}")
38     # tensor([[ 11.,   54.,   25.],
39     #         [ 10., -21.,   36.]])
40     x_re = dequantize_func(x_q_clip, S, Z)
41     print(f" 反量化后: \n{x_re}")
42     # tensor([[ -1.4118,  28.9412,   8.4706],
43     #         [-2.1176, -24.0000,  16.2353]])

```

9.3.2 大模型训练后量化方法

基于上述的量化基础知识，本部分将主要介绍大语言模型相关的量化方法。通常来说，模型量化方法可以分为两大类，即量化感知训练（Quantization-Aware Training, QAT）和训练后量化（Post-Training Quantization, PTQ）。从方法的名字可

以看出，量化感知训练方法需要更新权重进而完成模型量化，而训练后量化方法则无需更新模型权重。与小型语言模型相比，在为大语言模型设计或选择量化方法时需要侧重关注两个主要因素。首先，大语言模型由大规模神经网络参数组成，在设计量化算法时需要考虑到所需要花费的计算开销。一般来说，训练后量化方法需要更少的算力开销，实践中应用更为广泛。其次，大语言模型中具有不同的数值分布特征（如激活值中存在较大的数值），这使得对于大语言模型进行低比特量化变得非常困难，特别是对于激活值。下面将简要介绍几种具有代表性的大语言模型的训练后量化方法²。

权重量化

首先介绍主要面向模型权重的量化方法。其中，主流的权重量化方法通常是基于逐层量化的方法进行设计的，旨在通过最小化逐层的重构损失来优化模型的量化权重，可以刻画为： $\arg \min_{\hat{\mathbf{W}}} \|\mathbf{X}\mathbf{W} - \hat{\mathbf{X}}\mathbf{W}\|_2^2$ ，其中 $\mathbf{W}$ ， $\hat{\mathbf{W}}$ 分别表示原始权重和量化后的权重， $\mathbf{X}$ 为输入。

为了有效地优化该目标函数，GPTQ [246] 的基本想法是在逐层量化的基础上，进一步将权重矩阵按照列维度分组（例如 128 个列为一组），对一个组内逐列进行量化，每列参数量化后，需要适当调整组内其他未量化的参数，以弥补当前量化造成的精度损失。在具体的实现中，GPTQ 还进一步采用了特殊设计的优化方法来加速整个过程，如延迟批次更新、Cholesky 重构等。GPTQ 可以在 3 比特或 4 比特精度下实现对于大语言模型的有效权重量化。

进一步，AWQ [247] 发现在逐层和逐组权重量化过程中，对于模型性能重要的权重只占全部参数的一小部分（0.1%~1%），并且应该更加关注那些与较大激活值维度相对应的关键权重，因为它们对应着更为重要的特征。为了加强对于这部分关键权重的关注，AWQ 方法提出引入针对权重的激活感知缩放策略。具体来说，AWQ 的优化目标将逐层的重构损失 $\|\mathbf{X}\mathbf{W} - \hat{\mathbf{X}}\mathbf{W}\|_2^2$ 修改为： $\|(\text{diag}(\mathbf{s})^{-1} \cdot \mathbf{X}) \cdot \mathbf{Q}(\mathbf{W} \cdot \text{diag}(\mathbf{s})) - \mathbf{X}\mathbf{W}\|_2^2$ ，其中 $\mathbf{Q}$ 为量化函数。通过引入缩放因子 $\mathbf{s}$ ，AWQ 算法可以使得量化方法更为针对性地处理关键权重所对应的权重维度。

权重和激活值量化

下面继续介绍一些可以同时针对权重与激活值进行量化的方法。

- 细粒度量化. 对于 Transformer 模型来说，权重与激活值通常以张量的形式

²由于主要关注在大语言模型背景下的量化方法，因此本文未包括小型语言模型（例如 BERT）上的量化工作。

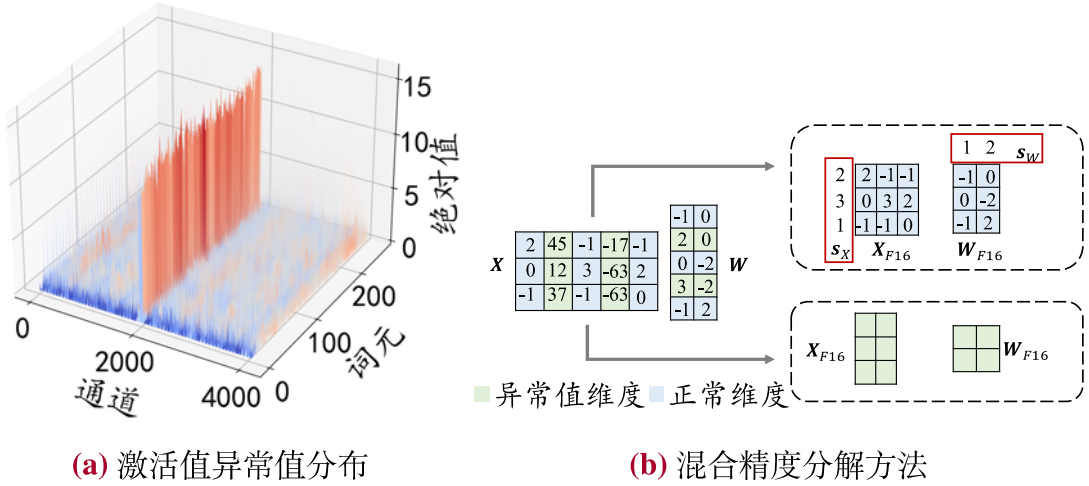


图 9.7 激活值异常值分布与混合精度分解

表示。如之前所述，可以使用粗粒度的方法量化，对于每个张量使用一整套量化参数。然而，这种粗粒度方法通常会导致不精确的数值重构，可以使用更为细粒度的方法来减小量化误差。举例来说，对权重量化，可以从每个张量改为对每一个通道使用一套量化参数。对于激活值量化来说，则是从每个张量改为对每个词元使用一套量化参数。ZeroQuant [248] 采用了一种带有动态校准的词元级量化方法来压缩激活值，而对于更容易量化的模型权重使用了基于分组的量化方法。在实际应用中，常用的模型量化分组可以设置为 128 [247, 248]。

- 混合精度分解. 相关研究发现 [249]，当模型参数规模超过一定阈值后（如 6.7B），神经网络中的激活值中会出现一些异常的超大数值，称为异常值涌现现象。有趣的是，这些异常值主要分布在 Transformer 层的某些特定激活值特征维度中。为了更好地理解异常值，图 9.7 (a) 中展示了 LLaMA (7B) 模型的激活值分布情况，可以观察到有一个通道（橙色）的绝对值明显高于其他通道，这就是前面提到的异常值维度。基于这一发现，在矩阵乘法中，可以将具有异常值的特征维度（橙色）与其他正常维度（蓝色）分开计算。如图 9.7 (b) 所示，对于这两部分进行计算时分别使用 16-比特浮点数 (F_{16}) 和 8-比特整数 (I_8)，从而以较高精度地恢复这些异常值。具体来说，对于非异常值维度部分，首先将激活值和权重分别量化到 8 比特整数，得到 X^{I_8} 和 W^{I_8} ，其量化过程如下所示：

$$127 \cdot \text{diag}(s_X)^{-1} \cdot X^{F_{16}} = X^{I_8}, \quad (9.9)$$

$$127 \cdot W^{F_{16}} \cdot \text{diag}(s_W)^{-1} = W^{I_8}, \quad (9.10)$$

其中， s_X 和 s_W 分别表示输入的激活值和权重中每一行/每一列中绝对值的最大值，

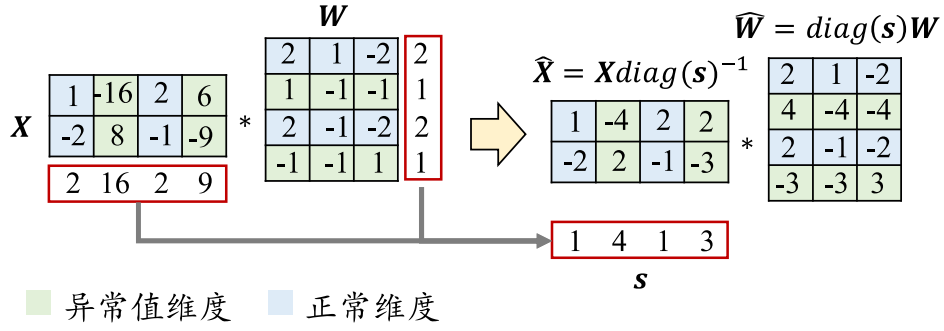


图 9.8 量化难度平衡方法

diag 表示将向量中的元素排列在对角线上，其他位置为零，形成对角矩阵。在量化数值 X^{I_8} 和 W^{I_8} 的基础上，可以进行 8 比特整数乘法操作，得到整数结果并用 32 位整数保存为 $O^{I_{32}}$ ，再进行反量化，得到 16 比特浮点数结果 $O^{F_{16}}$ 。图 9.7 (b) 展示了这一过程，具体的公式如下所示：

$$X^{I_8} W^{I_8} = O^{I_{32}}, \quad (9.11)$$

$$\frac{O^{I_{32}} \odot (s_X^T s_W)}{127 \cdot 127} = O^{F_{16}}, \quad (9.12)$$

其中， $\odot$ 表示矩阵逐元素相乘（哈达玛积）， $O^{I_{32}}$ 和 $O^{F_{16}}$ 分别表示整数和浮点数的结果。对于异常值维度部分，直接采用 16 比特浮点数乘法，过程如下所示：

$$X^{F_{16}} W^{F_{16}} = O^{F_{16}}. \quad (9.13)$$

这两部分各自得到的结果进行相加即可得到最终的结果。

• 量化难度平衡. 在模型的量化过程中，由于激活值中的异常值问题比权重更加明显，导致激活值往往比权重更加难量化。SmoothQuant [250] 提出将量化难度从激活值转移到模型权重上。具体来说，他们在线性层中引入了一个缩放变换来平衡权重和激活值之间的量化难度 $Y = (X \cdot \text{diag}(s)^{-1}) \cdot (\text{diag}(s) \cdot W)$ 。该公式通过数学上的等价变换，引入缩放因子向量 s 来控制量化难度。为了设置 s ，该公式还引入了一个强度迁移参数 α 来平衡量化难度，其中每个迁移系数的计算方法如下： $s_j = \max(x_j)^\alpha / \max(w_j)^{(1-\alpha)}$ 。具体来说，图 9.8 展示了这个变换的过程。这里以 $\alpha = 0.5$ 为例，则 s 的计算为 $s_j = \sqrt{\max(x_j) / \max(w_j)}$ 。从图中看出，变换后得到 $\hat{X}$ 中异常值维度明显缓解，而对应维度的权重 $\hat{W}$ 也会受到一些影响，但是相对于整体来说没有那么突出。

在实际应用中，上述所提出的量化策略可以联合使用，进而提高量化性能。此外，量化方法还依赖于硬件或系统级别的支持，如高效的 GPU 内核或硬件友好的

组划分方法，以保证算法的运行效率。

其他量化方法

在上述内容中已经主要介绍了训练后量化方法。下面将介绍面向大模型的微调增强量化方法与量化感知训练方法。

- **高效微调增强量化**. 对于大模型来说，直接进行低比特的量化（例如，INT4 量化）通常会导致性能的严重下降。为了克服这一挑战，QLoRA [216] 在量化模型中进一步引入了额外的小型可调适配器，并且使用 16 比特精度进行训练与学习，用以补偿量化可能带来的损失。这一思路同时结合了 LoRA 的轻量化优点（见第 7.3 节）与量化方法的模型压缩能力。实验结果表明，QLoRA 可以通过 4 比特量化模型很好地保留原来 16 比特模型的微调性能。

- **面向大语言模型的量化感知训练**. 由于量化感知训练方法需要引入额外的全参数学习，需要较大的计算开销，因此目前受到的研究关注还不多。最近一项研究工作测试了量化感知训练方法在大语言模型上的效果，通过教师模型生成的文本以及输出的预测分布来蒸馏小模型的方法，来压缩权重、激活值以及键值缓存 [251]。通过在开源模型 LLaMA 上进行实验，实验结果表明在权重和键值缓存上进行 4 比特量化可以获得不错的效果，但是在 4 比特激活值上的量化效果仍然需要进一步的提升。

基于开源代码库的 YuLan 模型量化实践

在本部分内容中，我们以 YuLan 模型为例，介绍基于开源量化代码库的大语言模型量化实践，主要介绍 bitsandbytes 和 GPTQ-for-LLaMA 两个代码库的使用。

- *bitsandbytes*³. bitsandbytes 基于 LLM.int8() [249] 和 8 比特优化器 [252] 论文中介绍的方法开发而成。该库主要专注于大语言模型的 INT8 量化，主要提供对 8 比特矩阵乘法和 8 比特优化器的支持。目前，bitsandbytes 还支持 4 比特的权重量化和混合精度分解方法，包括 NF4 (4-bit NormalFloat) 和 FP4 数据类型，可以进行加速模型的输出解码以及基于 QLoRA 的轻量化微调。在使用上，bitsandbytes 已经集成在 Hugging Face 中，可以在加载模型时直接通过运行参数指定实现对模型权重的量化。例如，可以使用参数 `load_in_8bit` 和 `load_in_4bit` 对模型进行 8 比特和 4 比特量化。

³<https://github.com/TimDettmers/bitsandbytes>

```

1 # bitsandbytes 实战
2 from transformers import AutoModelForCausalLM
3 name = "yulan-team/YuLan-Chat-2-13b-fp16"
4
5 # 8bit 模型量化
6 model_8bit = AutoModelForCausalLM.from_pretrained(name, device_map="auto",
7 ↪ load_in_8bit=True)
8 print(f"memory usage: {torch.cuda.memory_allocated()/1000/1000/1000} GB")
9
10 # 4bit 模型量化
11 model = AutoModelForCausalLM.from_pretrained(name, device_map="auto",
12 ↪ load_in_4bit=True)
13 print(f"memory usage: {torch.cuda.memory_allocated()/1000/1000/1000} GB")

```

- *GPTQ-for-LLaMA*⁴. 这个库专门用于量化 LLaMA 模型。它基于 GPTQ 算法 [246] 进行开发，可以用于各种参数规模大小的 LLaMA 模型（7B、13B 和 33B）进行 4 比特权重量化。该代码库的网站上提供了与 bitsandbytes 在显存和性能（困惑度）方面的比较，可供使用者进行对比与选择。关于计算资源，在使用该库的情况下，完成如 LLaMA (7B) 模型的解码实验只需要单张 RTX3090 (24G) 即可，能够有效降低模型对于显存资源的需求。

```

1 # GPTQ 实战
2 from transformers import AutoModelForCausalLM, AutoTokenizer, GPTQConfig
3 name = "yulan-team/YuLan-Chat-2-13b-fp16"
4
5 # 4bit 模型量化
6 tokenizer = AutoTokenizer.from_pretrained(name)
7 quantization_config = GPTQConfig(bits=4, dataset = "c4",
8 ↪ tokenizer=tokenizer)
9
10 model = AutoModelForCausalLM.from_pretrained(name, device_map="auto",
11 ↪ quantization_config=quantization_config)
12 print(f"memory usage: {torch.cuda.memory_allocated()/1000/1000/1000} GB")

```

9.3.3 经验性分析与相关结论

目前，模型量化已经成为一种重要的提升大模型部署效率的技术途径，能够显著减少显存资源占用以及解码延迟。在本部分内容中，我们首先总结学术界针对大模型量化的经验性分析结论，然后通过量化实验进一步探究了常用量化方法在不同精度下对于模型不同方面的性能影响。

⁴<https://github.com/qwopqwop200/GPTQ-for-LLaMa>

现有研究结论

针对大语言模型的模型量化研究受到了学术界的广泛关注，涌现了一批经验性的研究工作。下面针对学术界的研究结论进行一个简要汇总，使得读者更容易理解量化方法对于模型性能的影响以及不同量化方法的适用条件。

- *INT8* 权重量化通常对于大语言模型性能的影响较小，更低精度权重量化的效果取决于具体的量化方法 [246, 247, 250, 253]。在大多数情况下，*INT8* 权重量化可以有效地减小显存占用而不显著影响模型性能。对于 *INT4*（或 *INT3*）权重量化，现有的方法通常使用不同策略来减少性能下降。例如，*AWQ* 方法采用了激活感知缩放 [247]。与小型语言模型不同的是，低比特权重量化对于大语言模型的影响通常较小 [253]。因此，在实际使用中，在相同显存开销的情况下，建议优先使用参数规模较大的语言模型，而不是表示精度较高的语言模型。给定同一个系列的模型（如 *LLaMA* 系列），量化精度为 4 比特的 60GB 的语言模型在性能上往往会优于量化精度 8 比特的 30GB 的语言模型 [254]。此外，相关研究 [255] 还表明，经过 4 比特权重量化后，大语言模型的上下文学习能力、复杂推理能力和指令跟随能力受到的影响都很少。

- 对于语言模型来说，激活值相对于模型权重更难量化 [249, 250, 253]。如第 9.3.2 节所述，当 *Transformer* 语言模型的参数规模超过一个阈值后，激活值开始出现较大的异常值 [249]。数值较大的异常值对大语言模型激活量化带来了重要的挑战。为了克服这一问题，需要采用特定的处理方法，例如混合精度分解 [249]、细粒度量化 [249, 256] 和困难值迁移 [250] 来减轻异常值的影响，此部分内容可以参考第 9.3.2 节的内容。由于小型语言模型中激活值的范围通常相对标准，激活值量化对于小模型的模型效果影响较小 [253, 255]。尽管已经有一些研究工作表明，基于 *INT8* 的激活值量化可以获得较好的模型效果，但是这一任务仍然具有较高的研究挑战。此外，即使对于量化感知的训练方法 [251]，较低精度的激活值量化仍然需要更多探索。

- 轻量化微调方法可以用于补偿量化大语言模型的性能损失 [214, 216]。大语言模型在超低比特权重量化时（如 2 比特量化），可能会出现预测精度的大幅下降，这种精度下降问题可以通过轻量化微调（如 *LoRA* [214]）的方式来进行性能补偿。回顾第 7.3.1 节中介绍的 *LoRA* 方法，其核心是维护两部分参数，包括不微调的模型权重与微调的适配器参数。基于 *LoRA* 的性能补偿方法的基本想法是，针对模型权重进行低比特量化，而对于适配器参数则使用 16 比特浮点数表示并使用

LoRA 算法进行微调。在推理时，量化部分的模型权重会先反量化为 16 比特浮点数，再与适配器权重相加进行融合使用。相关研究表明 [255]，上述基于 LoRA 微调的性能补偿方法能够较好地恢复超低比特量化模型的性能：经过微调后，可以将 65B 参数模型在 2 比特权重量化的效果提升到接近 13B 模型的 16 比特精度。此外，QLoRA [216] 更为针对性地设计了面向量化模型的性能补偿方法，在轻量化微调的同时还考虑了显存优化，主要提出了三种改进方法，包括引入新的数据类型 NF4（4-bit NormalFloat）来缓解量化误差、提出双重量化技术以减少平均显存占用，以及分页优化器来管理显存峰值。实验表明 [216]，QLoRA 在基于 4 比特量化模型的微调后，能够获得与 16 比特模型全参数微调以及 LoRA 微调相似的效果。总结来说，通过轻量化微调来补偿量化大语言模型的精度损失，可以在模型效果和训练成本之间取得较好平衡，具有良好的应用前景。

实验性分析

为了帮助读者更好地理解量化方法对于语言模型的性能影响，本部分内容中展示了一系列相关实验结果，旨在检查指令微调模型在权重量化后的表现。这里使用 bitsandbytes 工具库对于经过指令微调的 LLaMA 模型进行权重量化，主要通过指定参数 `load_in_8bit` 和 `load_in_4bit` 对于模型进行量化。评测的模型包括 LLaMA (7B) 和 LLaMA (13B)，微调的数据集为三种广泛使用的指令微调数据集，包括 FLAN-v2 [41]、Alpaca-52K [42] 和 ShareGPT [38] 数据集。评测的参数精度为 4 比特、8 比特和非量化（16 比特）精度。

表 9.3 展现了 LLaMA 模型在三种不同量化精度下的模型性能，可以看到，使用 8 比特和 4 比特权重量化所获得的结果接近于原始 16 比特精度的模型性能。同时，由于使用了量化压缩，能够显著减少语言模型的显存开销。在实际应用中，如果显存资源比较受限，优先推荐尝试使用经过 4 比特权重量化的大语言模型。

9.4 其他模型压缩方法

除了模型量化之外，下面再介绍两种常见的模型压缩方法，即模型蒸馏和模型剪枝。与模型量化不同，模型蒸馏和模型剪枝则通过精简模型的结构，进而减少参数的数量。

表 9.3 不同表示精度的模型性能对比（表格来源：[10]）

模型	微调数据集	表示精度	AlpacaFarm	MMLU	BBH	显存 (GB)
LLaMA (7B)	FLAN-v2	FP16	6.65	47.34	35.05	12.58
		INT8	6.15	47.02	35.17	6.65
		INT4	7.83	46.23	34.77	3.94
	Alpaca-52K	FP16	32.55	40.87	33.66	12.58
		INT8	33.60	39.98	34.38	6.65
		INT4	29.57	39.24	32.80	3.94
	ShareGPT	FP16	72.05	41.30	32.90	12.58
		INT8	72.86	39.34	32.71	6.65
		INT4	70.31	40.08	32.11	3.94
LLaMA (13B)	FLAN-v2	FP16	8.14	51.67	41.46	24.40
		INT8	7.64	51.02	41.25	12.53
		INT4	7.52	50.48	40.68	7.34
	Alpaca-52K	FP16	33.60	47.63	36.10	24.40
		INT8	31.43	47.04	35.98	12.53
		INT4	30.87	46.20	36.16	7.34
	ShareGPT	FP16	75.59	47.58	38.00	24.40
		INT8	73.79	47.71	38.31	12.53
		INT4	71.99	45.77	36.97	7.34

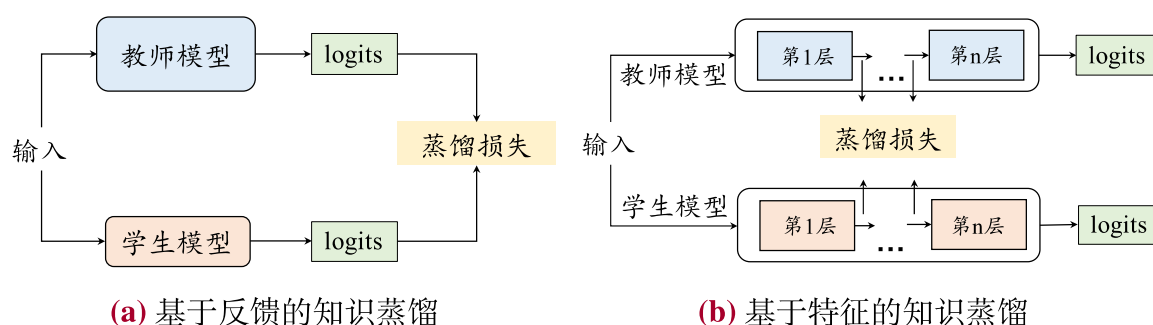


图 9.9 基于反馈的知识蒸馏和基于特征的知识蒸馏对比

9.4.1 模型蒸馏

模型蒸馏 (Model Distillation) 的目标是将复杂模型 (称为教师模型) 包含的知识迁移到简单模型 (称为学生模型) 中, 从而实现复杂模型的压缩。一般来说, 通常会使用教师模型的输出来训练学生模型, 以此来传递模型知识。以分类问题为例, 教师模型和学生模型在中间每一层会输出特征表示 (特指神经网络模型), 在最后一层会输出针对标签集合的概率分布。模型蒸馏的核心思想是, 引入额外的损失函数 (称为蒸馏损失函数), 训练学生模型的输出尽可能接近教师模型的输

出。在实际应用中，蒸馏损失函数通常与分类损失函数（交叉熵损失函数）联合用于训练学生模型。下面首先介绍传统的知识蒸馏方法，再介绍其在大语言模型中的应用。

传统的知识蒸馏方法

根据蒸馏知识的不同，传统的模型蒸馏方法包括两种类型：基于反馈的知识蒸馏方法（图 9.9 (a)）和基于特征的知识蒸馏方法（图 9.9 (b)）。

- 基于反馈的知识蒸馏. 这种方法主要关注教师模型最后一层输出的 logits，这些 logits 经过 softmax 变换后，可以用作学生模型的“软标签”来进行学习，如图 9.9 (a) 所示。蒸馏损失函数可以形式化表示为：

$$\mathcal{L}(l_t, l_s) = \mathcal{L}_R(p_t(\cdot), p_s(\cdot)), \quad (9.14)$$

其中 $\mathcal{L}_R$ 往往使用 KL 散度作为衡量指标， l_t, l_s 分别表示教师模型和学生模型的输出 logits， $p_t(\cdot)$ 和 $p_s(\cdot)$ 分别表示教师模型和学生模型的 logits 经过 softmax 函数所获得的预测标签概率分布。从公式 9.14 可以看出，优化的核心目标是让学生模型输出的 logits 去近似教师模型输出的 logits，进而通过这种方式让学生模型学习到教师模型的特有知识。

- 基于特征的知识蒸馏. 与基于预测分布的蒸馏相比，基于中间特征表示的蒸馏（如图 9.9 (b) 所示）关注于教师模型的中间层输出的激活值，并使用这些激活值作为监督信息训练学生模型。例如，在基于多层 Transformer 架构的大语言模型中，每一层输出的特征表示都可以用作知识。相应的蒸馏损失函数可以表示为：

$$\mathcal{L}(f_t(x), f_s(x)) = \mathcal{L}_F(\Phi(f_t(x)), \Phi(f_s(x))), \quad (9.15)$$

其中 $f_t(x)$ 和 $f_s(x)$ 分别表示教师模型和学生模型的中间层输出特征， $\Phi(\cdot)$ 表示变换函数用于处理输出形状不匹配的情况， $\mathcal{L}_F(\cdot)$ 是一个相似度函数，用于衡量教师模型的中间层特征与学生模型的中间层特征的相似度。

相较于最终的预测分布，中间层特征提供了更为丰富的模型信息，有助于在模型蒸馏过程中实现更为有效的知识迁移。然而，这种方法也存在一些技术难点，如消除架构不一致的影响、目标层自动化选择等。

大语言模型的知识蒸馏方法

面向大语言模型的知识蒸馏旨在将大语言模型（教师模型）包含的知识迁移到小模型（学生模型）中。根据大语言模型权重是否可以获得，可以分别使用白盒模型蒸馏方法和黑盒模型蒸馏方法。其中，白盒模型蒸馏方法可以获取模型的

权重来指导学生模型，典型的方法为 MINILLM [257]，其最大可将基于 LLaMA 结构的 13B 参数模型蒸馏到 7B；而黑盒模型蒸馏方法无法获得模型权重，只能使用其输出信息来训练小模型。目前，比较典型的工作主要关注于蒸馏大模型的关键能力，如上下文学习能力、思维链推理能力以及指令遵循能力。下面以思维链推理能力的蒸馏为例进行介绍（关于思维链的具体介绍参考第 10.3 节）。

问题: A gentleman is carrying equipment for golf, what is he likely to have?

选项: (a) club (b) assembly hall (c) meditation center (d) meeting (e) church

思维链: The answer must be something that is used for golf. Of the above choice, only clubs are used for golf. So the answer is (a) club

例 9.2 思维链能力的知识蒸馏输入示例

为了能够充分蒸馏思维链推理能力，可以使用大语言模型所生成的支持其预测结果的思维链推理文本 [258]。具体来说，可以利用大语言模型生成输入样本的思维链推理过程，作为除标签以外的额外补充信息，来帮助引导小模型学习。这种方法通过引入大模型对于问题的求解思路来提供额外的监督信息。这里以一个选择题为例进行介绍。如例 9.2 所示，其中输入为“问题”和“选项”，然后使用大模型生成对应的解释“思维链”，其中正确的答案为“(a) club”。在这个输入的基础上，可以让学生模型同时学习预测标签，以及学习大模型生成的对应的推理过程解释。上述思路可以形式化表示为下面的损失函数：

$$\mathcal{L} = \mathcal{L}_{\text{label}} + \lambda \mathcal{L}_{\text{cot}}, \quad (9.16)$$

其中， $\mathcal{L}_{\text{label}}$ 表示对于标签的预测损失， $\mathcal{L}_{\text{cot}}$ 表示生成思维链文本所带来的损失， λ 为结合系数。根据实验测试，这种蒸馏方法在训练特定任务的小模型时，可以有效降低数据的使用量：在 ANLI 数据集上微调 770M 参数的 T5 模型，仅需要 80% 样本就可以达到标准微调全部样本的效果。

9.4.2 模型剪枝

模型剪枝 (Model Pruning) 的目标是，在尽可能不损失模型性能的情况下，努力消减模型的参数数量，最终有效降低模型的显存需求以及算力开销。这里主要从传统模型剪枝方法和大模型剪枝方法两个方面来介绍。

传统模型剪枝方法

传统模型剪枝方法一般可以被分为两类, 包括结构化剪枝和非结构化剪枝。下面针对这两种剪枝方法进行介绍。

- 结构化剪枝. 结构化剪枝 (Structured Pruning) 旨在去除对于性能影响较小的模型组件, 可以删除神经元、通道甚至中间层。一般来说, 大语言模型都是由多个 Transformer 层叠加而成, 具体对于模型结构的描述可以参阅第 5 章的介绍。结构化剪枝的核心思想是, 在尽量保持模型预测精度的条件下, 去除那些对于结果影响不大的结构单元, 如注意力机制中的注意力头、前馈层权重中的特定维度等。这里以前馈层中的维度剪枝为例进行介绍。具体来说, 通过计算每一维列向量权重的数值大小来作为判断其重要性的标准, 然后再去掉那些重要性较低的维度从而实现剪枝。在实践中, 可以采用 $L2$ 范数去度量权重的重要性。在下面的代码实现中, 以 $L2$ 范数为重要性度量进行结构化剪枝。在 PyTorch 中通过调用 `torch.nn.utils.prune` 库中的 `ln_structured` 函数, 具体的代码如下所示:

```
1 # module.weight 为待剪枝的权重, amount 为要剪枝的比例,
2 # n=2 表示采用  $L2$  norm, dim=0 表示剪枝的是第 0 维度
3 prune.ln_structured(module, name="weight", amount=0.5, n=2, dim=0)
```

- 非结构化剪枝. 作为另一种剪枝方法, 非结构化剪枝 (Unstructured Pruning) 主要关注去除模型权重矩阵中不重要的数值。与结构化剪枝不同, 非结构化剪枝并不修改模型的结构。一般来说, 模型权重都是以矩阵形式进行表示与存储的, 非结构化剪枝通过创建一个包含 0/1 的掩码矩阵, 并将这一矩阵与原始的权重相乘, 其中 0 所在位置的权重则不会在模型的计算中产生作用。当剪枝完成后, 那些被剪枝掉的位置只会存储数值 0, 从而节省了存储空间。在下面的代码实现中, 为了进行权重剪枝 (以 30% 为例), 使用了 $L1$ 范数来评估参数的重要性, 在 PyTorch 中通过调用 `torch.nn.utils.prune` 库中的 `l1_structured` 函数来实现这一目标:

```
1 # module.weight 为待剪枝的权重, amount 为要剪枝的比例,
2 # n=2 表示采用  $L2$  norm, dim=0 表示剪枝的是第 0 维度
3 prune.l1_structured(module, name="weight", amount=0.3)
```

总体来说, 在实际应用中, 非结构化剪枝一般可以实现更高的剪枝比率, 但是不会显著加速模型的计算过程, 因为被掩码的部分可能仍然需要参与计算。要想实现加速, 则需要特定软件和硬件的支持 [259]。而结构化剪枝通过去除结构单

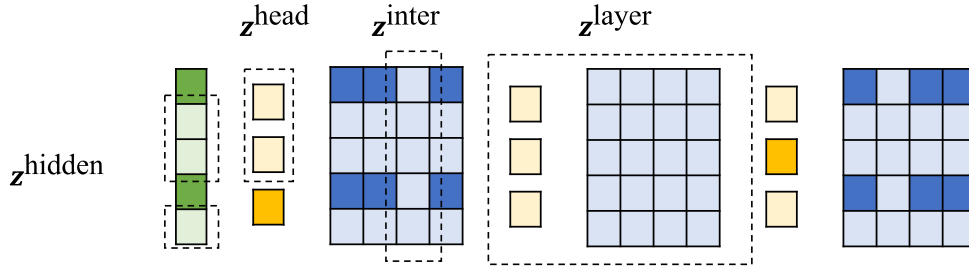


图 9.10 大语言模型的剪枝方法—sheared LLaMA 示意图

元，可以显著减少所需的矩阵乘法次数，实现模型的压缩和加速。

大语言模型的剪枝方法

面向大语言模型的剪枝目的是在尽可能少地影响模型性能的情况下减少其计算资源的需求。与传统的模型剪枝类似，主要分为结构化和非结构化剪枝两类。其中，非结构化剪枝一般容易获得更高的压缩率，典型的工作包括 SparseGPT [260]，其只需要使用 1 张 A100 (80G) 显卡就可以完成对于 175B 参数规模大语言模型（如 OPT 模型）的剪枝，实现 60% 模型参数的剪枝，并较好地保持了困惑度不升。作为另一类方法，面向大模型的结构化剪枝研究也取得了较好的模型压缩效果。例如，LLM-prune [261] 在 LLaMA (7B) 上实现了剪枝 20% 参数但是依然保持原始模型 93.6% 的预测精度。Sheared LLaMA [262] 则将 LLaMA-2 (7B) 剪枝得到 2.7B 参数规模，保持原始模型 87.8% 的预测精度。下面以 Sheared LLaMA 为例详细介绍大语言模型的剪枝算法。

Sheared LLaMA [262] 将结构化剪枝问题转化为一个约束优化问题，旨在得到针对不同结构的掩码变量用于去除不重要的参数（如注意力头、特征维度等），使得未被掩码部分构成的子网络能够获得较优的效果。如图 9.10 所示，这里以注意力头为例，掩码变量为 $\mathbf{z}^{\text{head}} \in \mathbb{R}^{N_S}$ ，其中 N_S 表示源模型注意力头总数，其内部每个元素的值为 0 或 1，其中第 i 个元素的值为 0 表示该注意力头将被删除。

为了搜索需要被剪枝的结构，可以使用拉格朗日乘子法将剪枝的目标表示为损失函数中的约束条件，在优化过程中对于剪枝比例和模型性能之间进行权衡，从而实现特定结构的剪枝。例如，针对单层 Transformer 注意力头的约束可以表示为：

$$\tilde{\mathcal{L}}^{\text{head}}(\lambda, \phi, \mathbf{z}) = \lambda^{\text{head}} \cdot (\sum \mathbf{z}^{\text{head}} - N_{\mathcal{T}}) + \phi^{\text{head}} \cdot (\sum \mathbf{z}^{\text{head}} - N_{\mathcal{T}})^2, \quad (9.17)$$

其中 $N_{\mathcal{T}}$ 表示目标结构的注意力头数。类似地，针对中间维度约束可以表示为 $\tilde{\mathcal{L}}^{\text{int}}$ ，针对隐藏层维度约束为 $\tilde{\mathcal{L}}^{\text{hidden}}$ ，针对层数的约束为 $\tilde{\mathcal{L}}^{\text{layer}}$ 。基于这个约束条件，模

型剪枝的目标函数可以被表示为联合优化下式，即 $\min_{\theta, z} \max_{\lambda, \phi} \mathcal{L}^{\text{prune}}(\theta, z, \lambda, \phi)$ ：

$$\min_{\theta, z} \max_{\lambda, \phi} \mathcal{L}^{\text{prune}}(\theta, z, \lambda, \phi) \quad (9.18)$$

$$= \min_{\theta, z} \max_{\lambda, \phi} \mathcal{L}(\theta, z) + \sum_{j=1}^{L_S} \tilde{\mathcal{L}}_j^{\text{head}} + \sum_{j=1}^{L_S} \tilde{\mathcal{L}}_j^{\text{int}} + \tilde{\mathcal{L}}^{\text{layer}} + \tilde{\mathcal{L}}^{\text{hidden}}, \quad (9.19)$$

其中， $\mathcal{L}(\theta, z)$ 表示使用该掩码变量 z 进行剪枝时语言模型的损失， L_S 表示总层数。从上式可以看到，模型剪枝被转化为在给定的训练数据集上的优化问题，通过学习满足优化目标的掩码变量来删除或保留特定的结构。

为了更好地对于上述目标进行优化，Sheared LLaMA 进一步提出了动态数据配比的学习方法。通常来说，预训练数据由多个领域的数据集组成，训练过程中将按照固定的数据配比进行采样。动态数据配比方法则是在训练过程中动态确定不同领域数据集的配比，以增加未较好习得的数据集的比重，减少已较好习得的数据集的比重，从而提升训练过程的数据学习效率。具体来说，在训练过程中，每隔一定步数后计算每个领域数据集上损失函数（如可以使用困惑度等指标）的实际值与目标值（可通过扩展法则进行预测 [22]）的差异来判断对当前领域数据集的掌握程度，进而对于数据集权重进行动态调整。

通过以上剪枝策略和继续预训练方法相结合，Sheared LLaMA 将 LLaMA-2 (7B) 模型剪枝到 2.7B 参数规模，并在 50B 词元的继续预训练后，最终恢复到原始模型 87.8% 的预测精度。